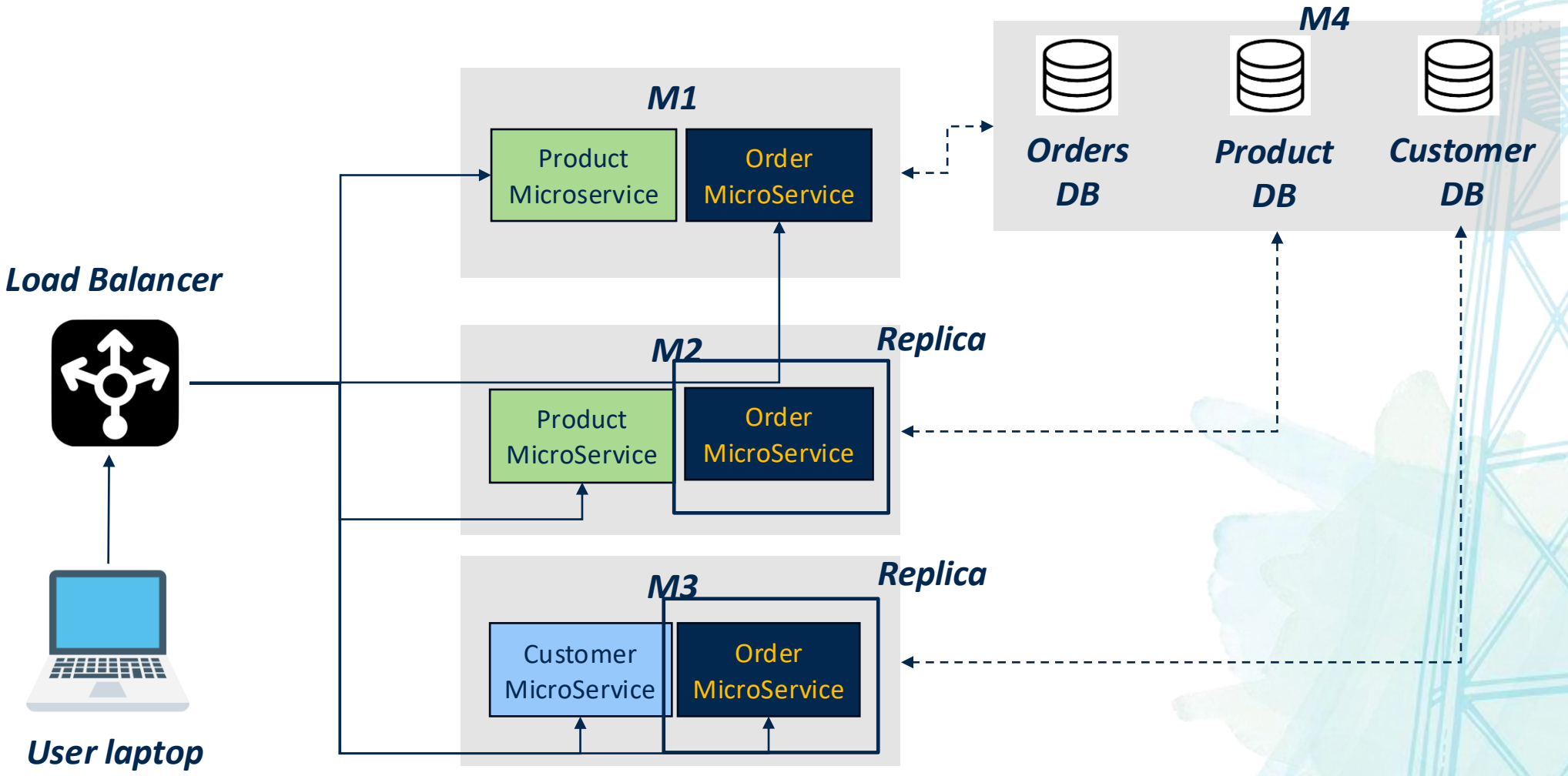
Modern application deployment

Tapti Palit

Modern application deployment

- Apps are deployed across many machines
- Many (micro-)services are replicated across many machines for improved throughput and redundancy

Online shop microservices



Automated cluster management

- Microservices deployed on hundreds of nodes
- Manually configuring and monitoring hundreds or thousands of nodes is not feasible
- Cluster management must be automated

Cluster management requirements

- Automatically schedule services on nodes
  - Each service needs its own environment
- Minimize communication overhead
  - Deploy ”chatty” microservices on the same node
- Ensure the node is well-provisioned to handle the service scheduled on it
  - Heterogenous clusters might have nodes with different CPU/RAM/disk capabilities
- Detect node failures to stop routing requests to the failed nodes

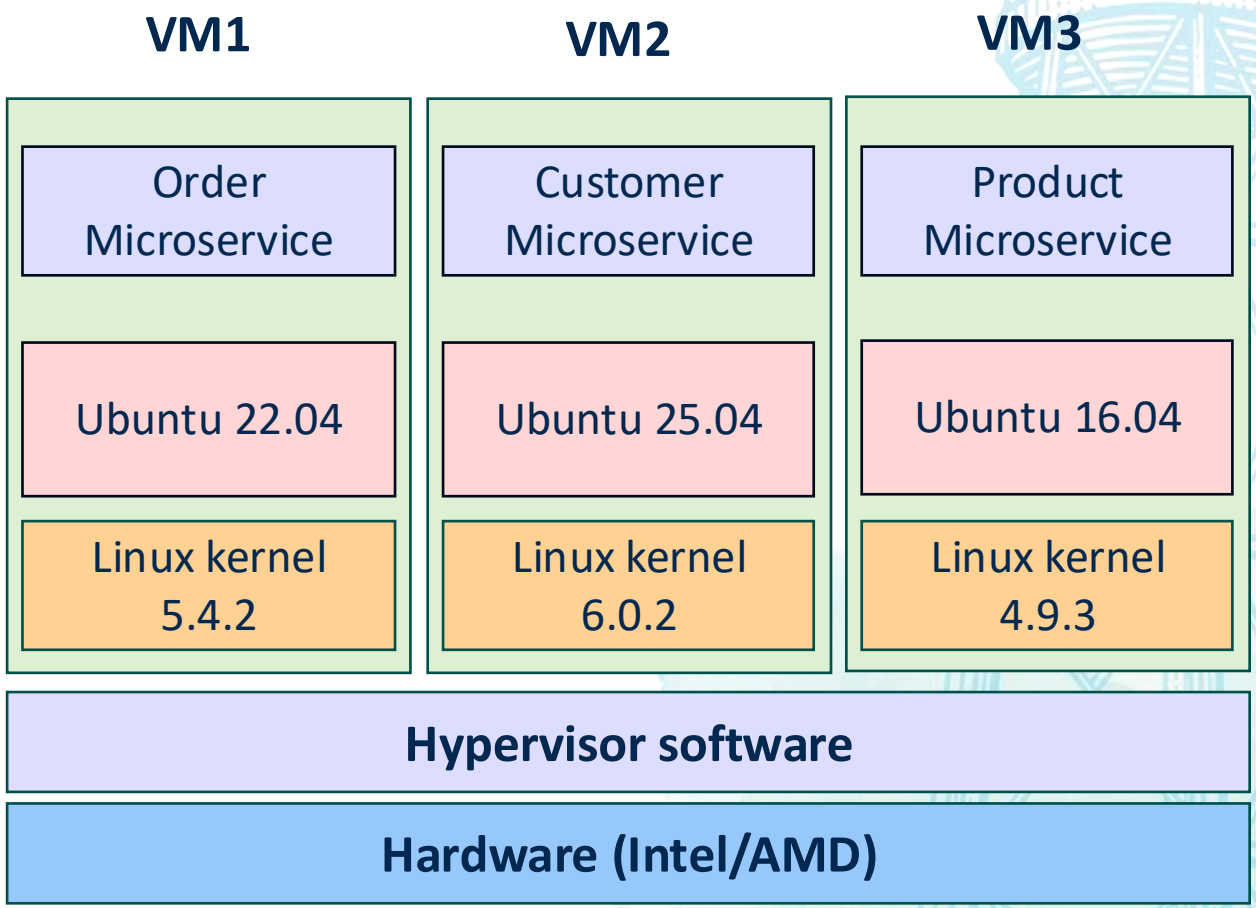
Kubernetes: goal & history

- Goal: automate deployment, scaling, and management of containerized applications
- Origin: born from Google's internal systems Borg and Omega
- Kubernetes (K8s) open-sourced in 2014

Virtual machines and containers

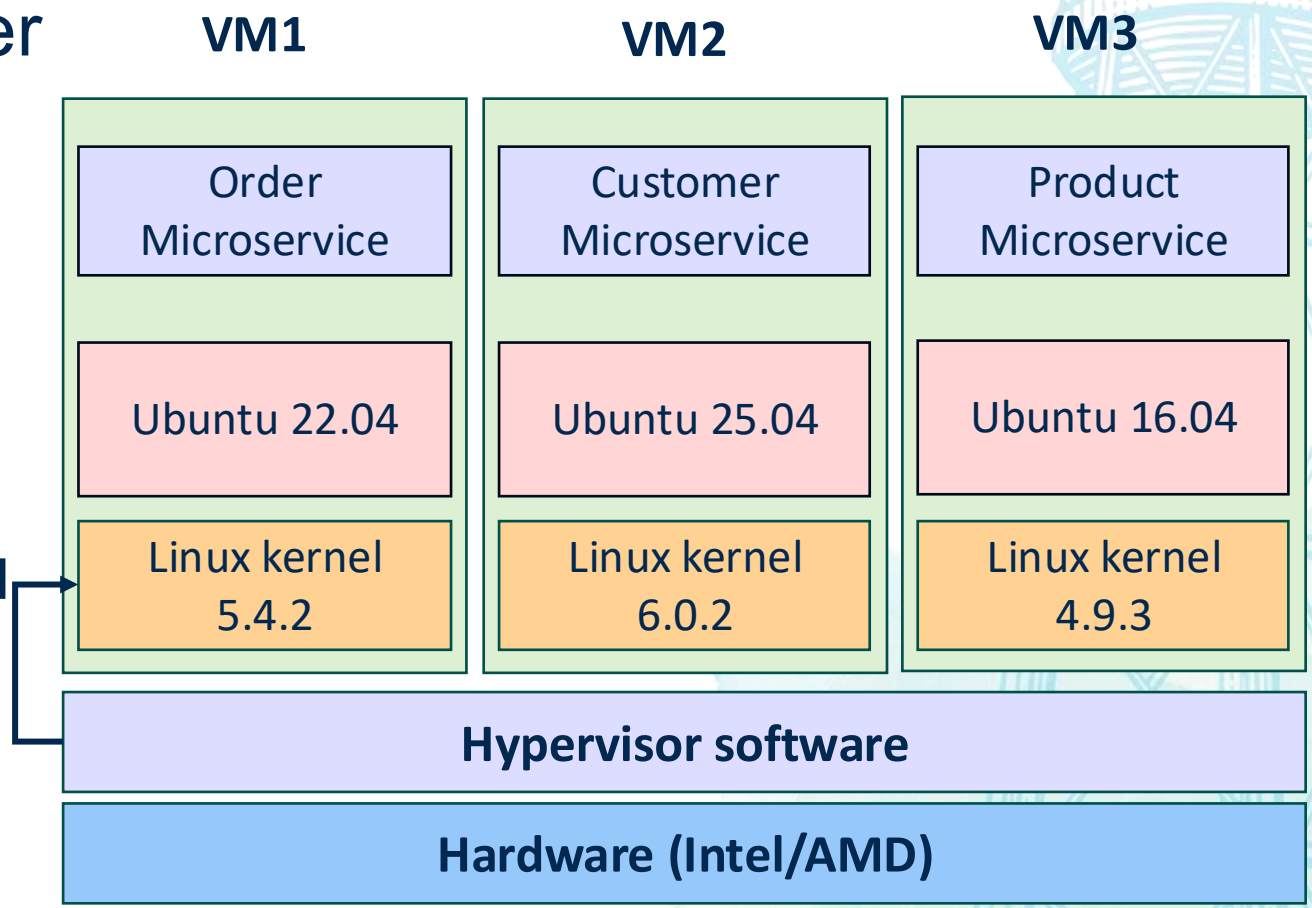
What is a virtual machine?

- Isolated environment that emulates a physical machine
- A hypervisor runs on the hardware
- Each VM has its own kernel
- Isolation enforced by hardware
  - Intel VT-x or AMD-V



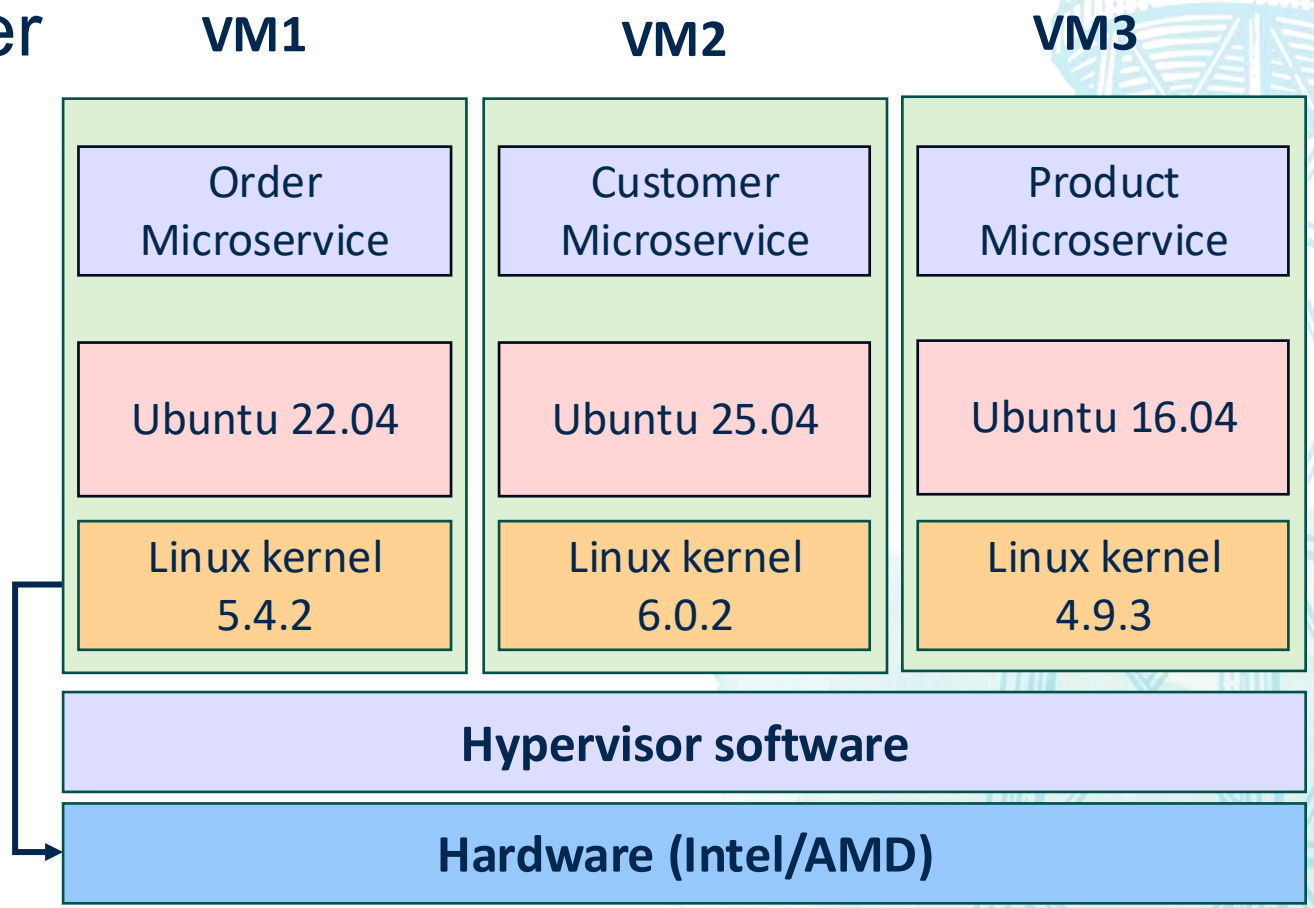
What is a virtual machine?

- Special CPU instructions to enter and exit VMs
  - VMRUN to enter “guest mode” and run the VM
  - VMCALL to exit “guest mode” and execute host code
- Expensive instructions and also cause cache, TLB invalidations



What is a virtual machine?

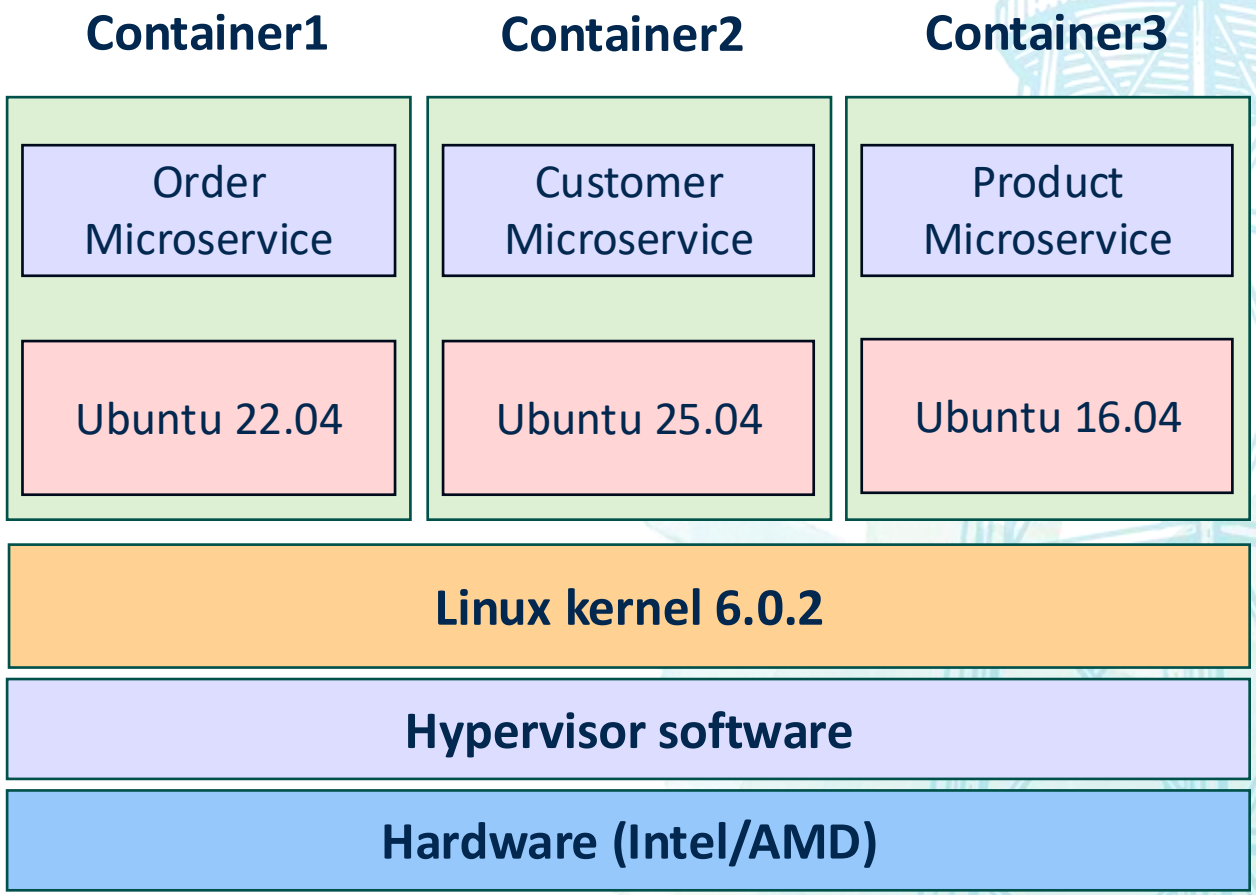
- Special CPU instructions to enter and exit VMs
  - VMRUN to enter “guest mode” and run the VM*
  - VMCALL to exit “guest mode” and execute host code
- Expensive instructions and also cause cache, TLB invalidations



* For AMD machines, Intel uses VMENTER, VMEXIT

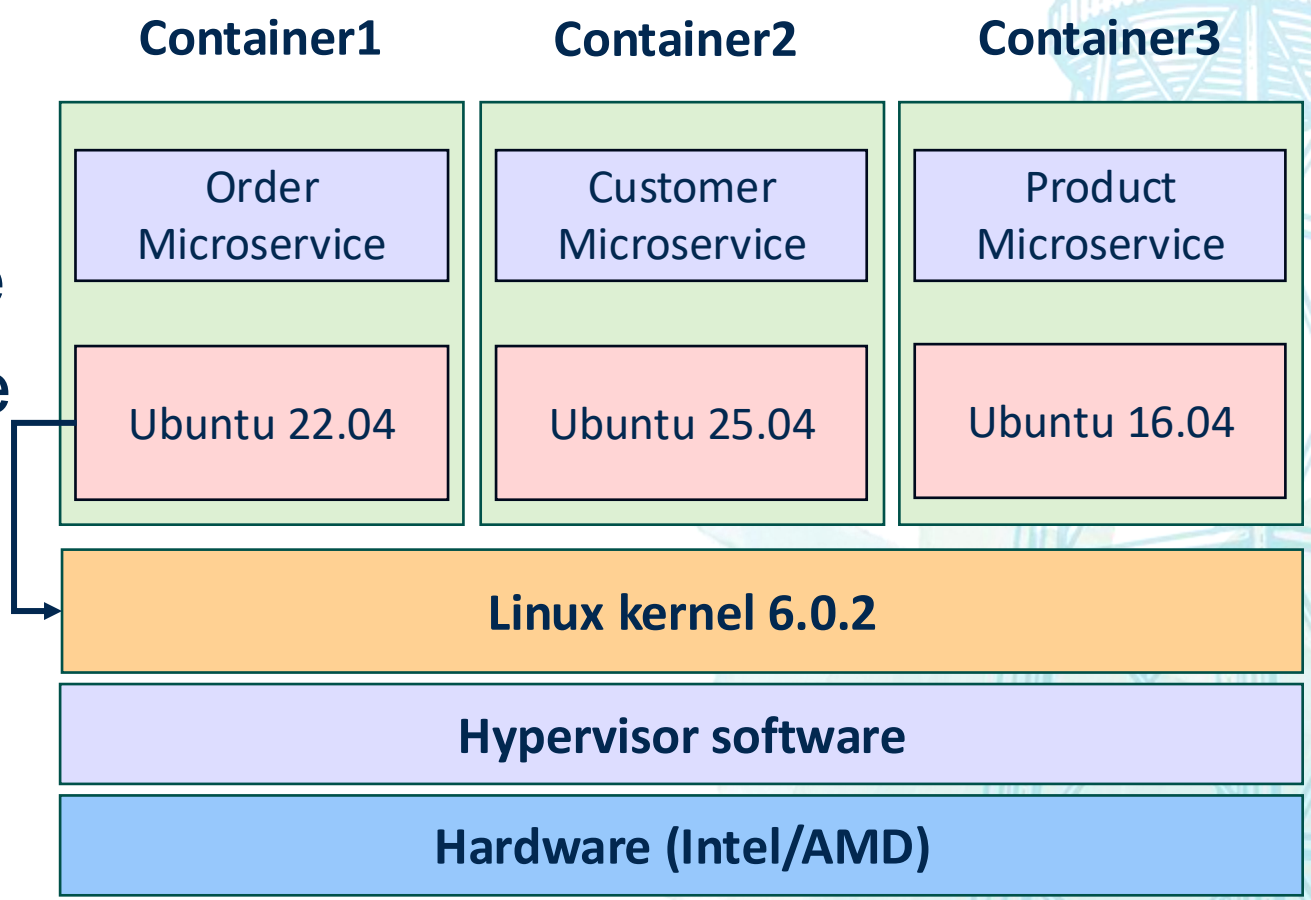
What is a container?

- Isolated user-space environment
- Each container shares the host kernel
- Isolation enforced by software
  - Namespaces and cgroups in the Linux kernel



What is a container?

- Namespaces partition kernel resources in software
- E.g., a process can only see the resources in its own namespace




```
ssize_t sys_read(int fd, void *buf, size_t n)
{
  // check if we're reading proc dir
  for_each_process(task) {
    if(!task_in_my_ns(task)) continue;
    // return the task details
  }
}
```

Tradeoffs

Virtual Machines	Containers
Full OS per VM (guest kernel)	Share the host kernel (not possible to run a containerized Windows app on Linux kernel)
Hardware-based isolation	Kernel-based isolation (namespaces + cgroups)
Heavy: GBs of memory overhead	Lightweight: MBs of overhead
Slow startup (minutes)	Fast startup (seconds)
Strong isolation boundary (better security)	Weaker isolation (shared kernel attack surface)
Commonly used in multi-tenant setups	Commonly used in single-tenant setups

Docker containers

- Docker is a tool for running software in containers
  - Declarative language for specifying what software should be packaged in a container
  - Commands to run the software as a container
- Linux kernel provides system call flags for creating new namespaces (and cgroups)
  - For e.g., `clone(child_fn, stack, CLONE_NEWUTS | CLONE_NEWPID | CLONE_NEWNS | SIGCHLD, arg);`
- Docker provides wrappers around these system calls

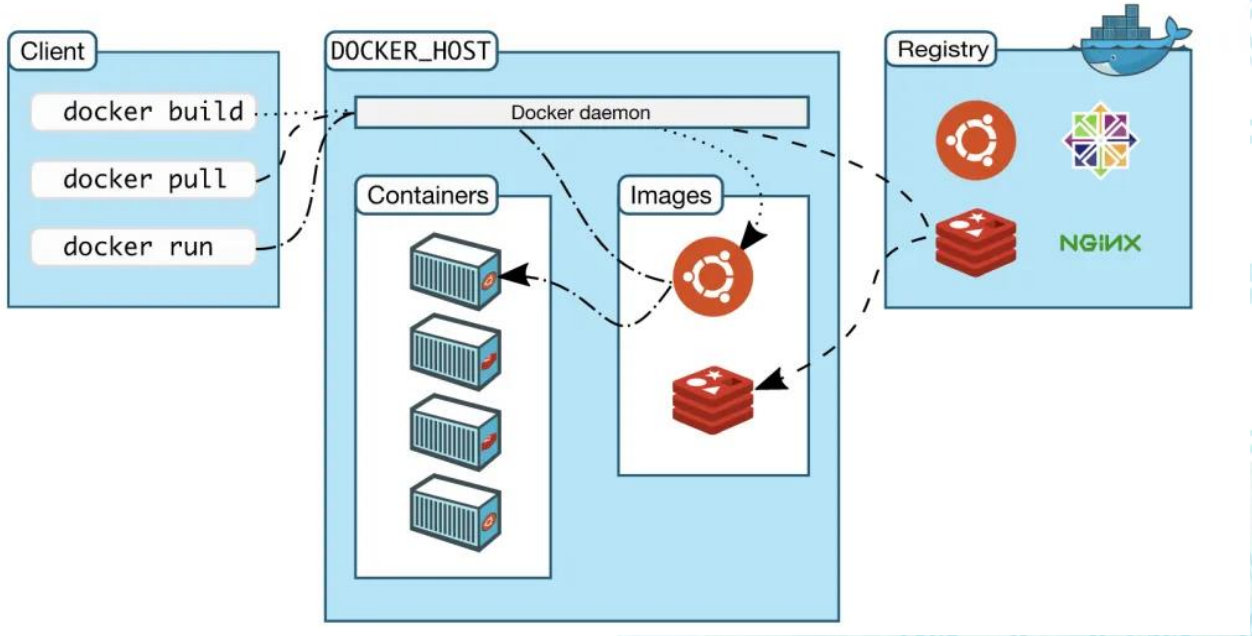
Sample Dockerfile

```
# Use the Ubuntu base image
FROM ubuntu:25.04

# Set the working directory
WORKDIR /app

ENV DEBIAN_FRONTEND=noninteractive
# Install required dependencies: OpenJDK, Maven, Git
RUN apt-get update && apt-get install -y \
    openjdk-21-jdk \
    maven \
    git \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*

# Set JAVA_HOME environment variable
ENV JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64
ENV PATH="$JAVA_HOME/bin:$PATH"

# Clone the Java application repository
RUN git clone https://github.com/davsec-teaching/junit_demo.git .

# Build the application using Maven
RUN mvn clean package

RUN mvn clean test surefire-report:report
# Define the command to run the application
CMD ["bash"]
```

```
# To build the docker container
➢  docker build -t java-app .

# To run the docker container
> docker run –it -p 8080:8080 java-app bash

# In the Docker container
> cat /etc/os-release
> top
> Who

# To push the docker container to remote registry
> docker push java-app:1.0
```

Docker: the container runtime

- Remote docker registry
  - Dockerhub contains container images
  - Each container image is user-space software bundled together into a filesystem
- Docker host
  - Machine running docker daemon
  - Stores docker images locally
  - Spawns containers



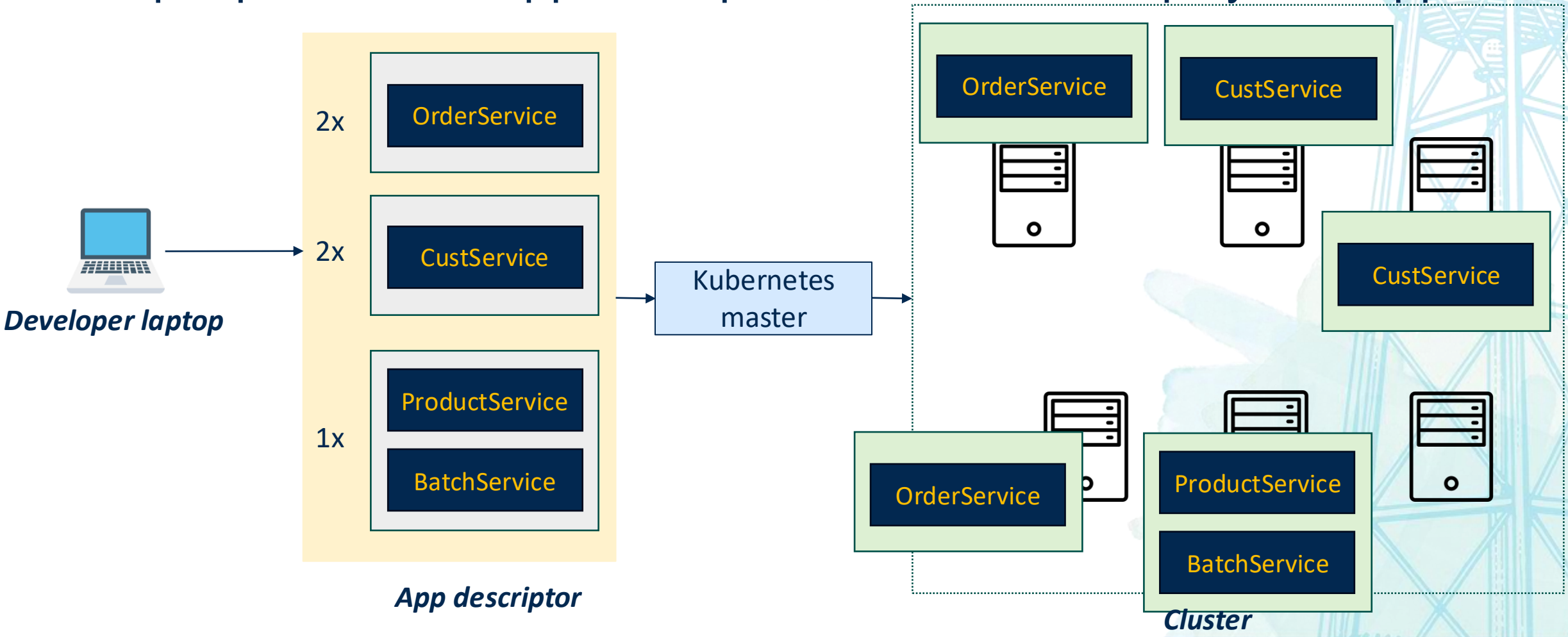
CI/CD

- Continuous integration (CI)
  - Developers merge changes frequently; every change triggers automated builds + tests to catch breakages early
  - E.g., GitHub Actions
- Continuous deployment (CD)
  - Passing changes are auto-deployed to production
- Containers provide uniform build system for CI/CD
  - No weird issues with `$PATH` variables
  - No missing binaries

Kubernetes

Kubernetes overview

Developer provides an app description; Kubernetes deploys the app



App descriptor

- Dev creates app descriptor in an YAML format
- `order-service`, `product-batch-service` are deployment objects
- Submit it to Kubernetes using
  - `kubectl apply -f app.yaml`
  - `kubectl get pods`

```yaml
apiVersion: apps/v1
metadata:
  name: order-service
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: order-service
          image: your-registry/orderservice:latest
---
apiVersion: apps/v1
metadata:
  name: product-batch-service
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: product-service
          image: your-registry/productservice:latest
        - name: batch-service
          image: your-registry/batchservice:latest
```

The pod abstraction

- A “pod” is a group of tightly related containers
- Each pod is separated logically with its own IP
- Pods are units of scaling – you scale by adding or removing pods

Pod-1

```
apiVersion: apps/v1
metadata:
  name: order-service
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: order-service
          image: your-registry/orderservice:latest
---
```

Pod-2

```
apiVersion: apps/v1
metadata:
  name: product-batch-service
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: product-service
          image: your-registry/productservice:latest
        - name: batch-service
          image: your-registry/batchservice:latest
```

The pod abstraction

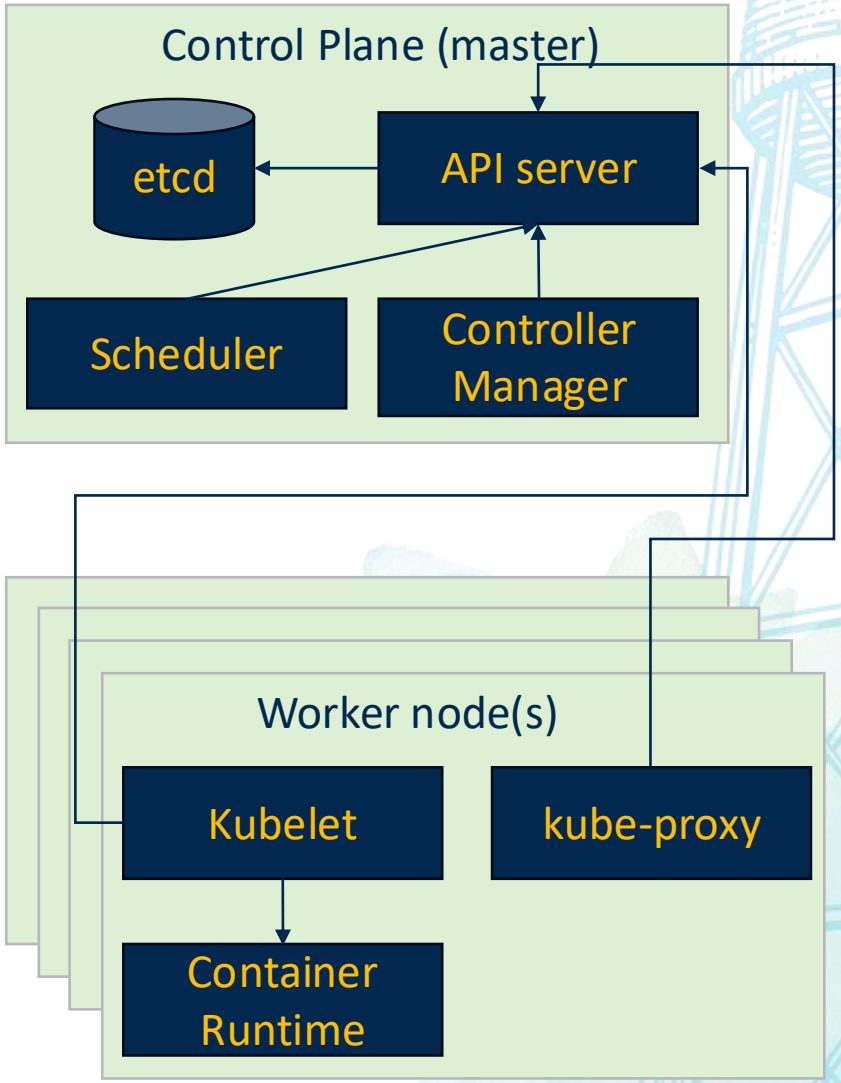
- Containers in each pod are guaranteed to be scheduled on the same machine
- Dev can schedule “chatty” microservices in same pod

```yaml
apiVersion: apps/v1
metadata:
  name: order-service
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: order-service
          image: your-registry/orderservice:latest
---
apiVersion: apps/v1
metadata:
  name: product-batch-service
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: product-service
          image: your-registry/productservice:latest
        - name: batch-service
          image: your-registry/batchservice:latest
```

Chatty microservices

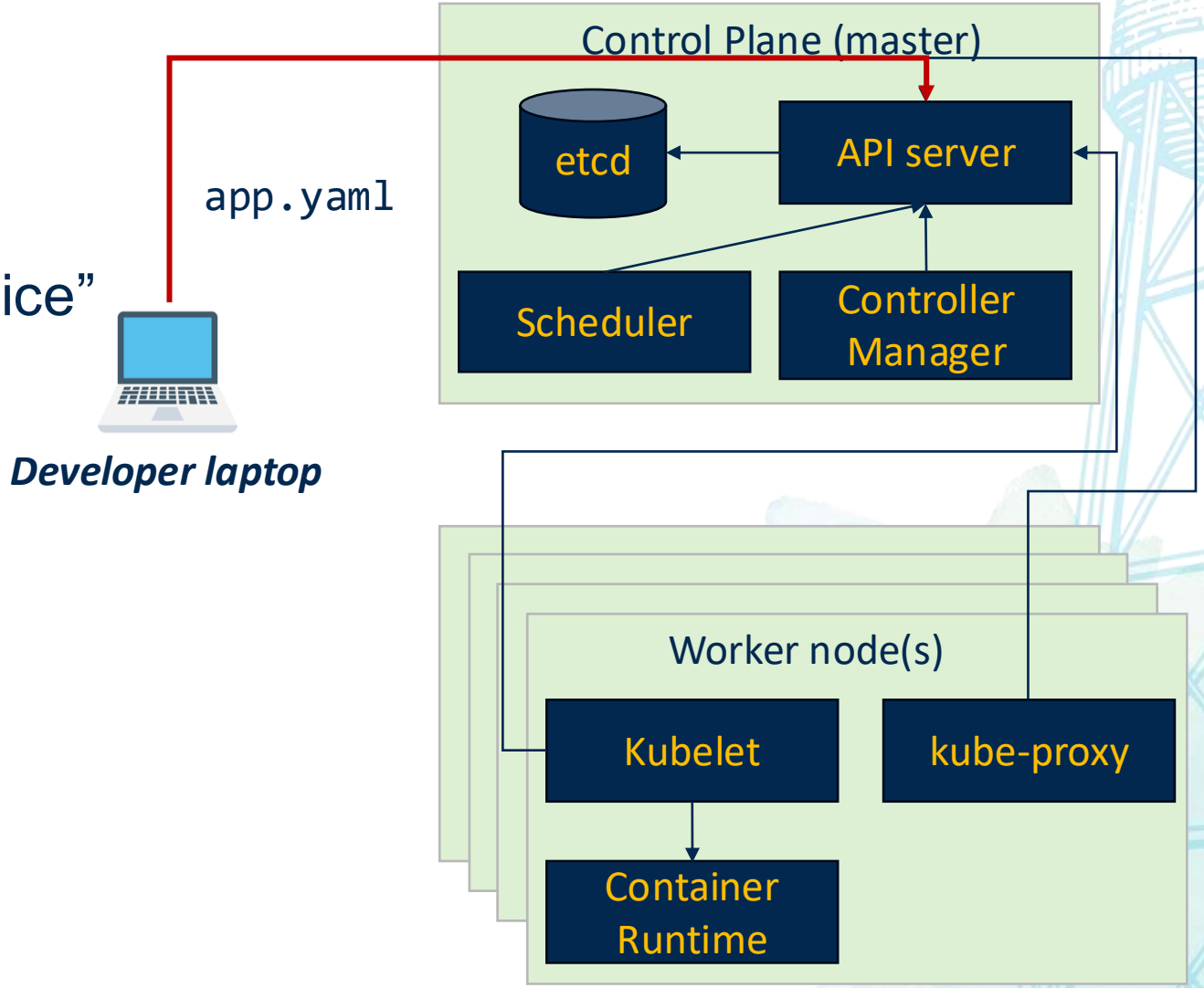
Kubernetes components

- Master node:
  - etcd, API server, Scheduler, Controller Manager
- Worker node
  - Kubelet, kube-proxy



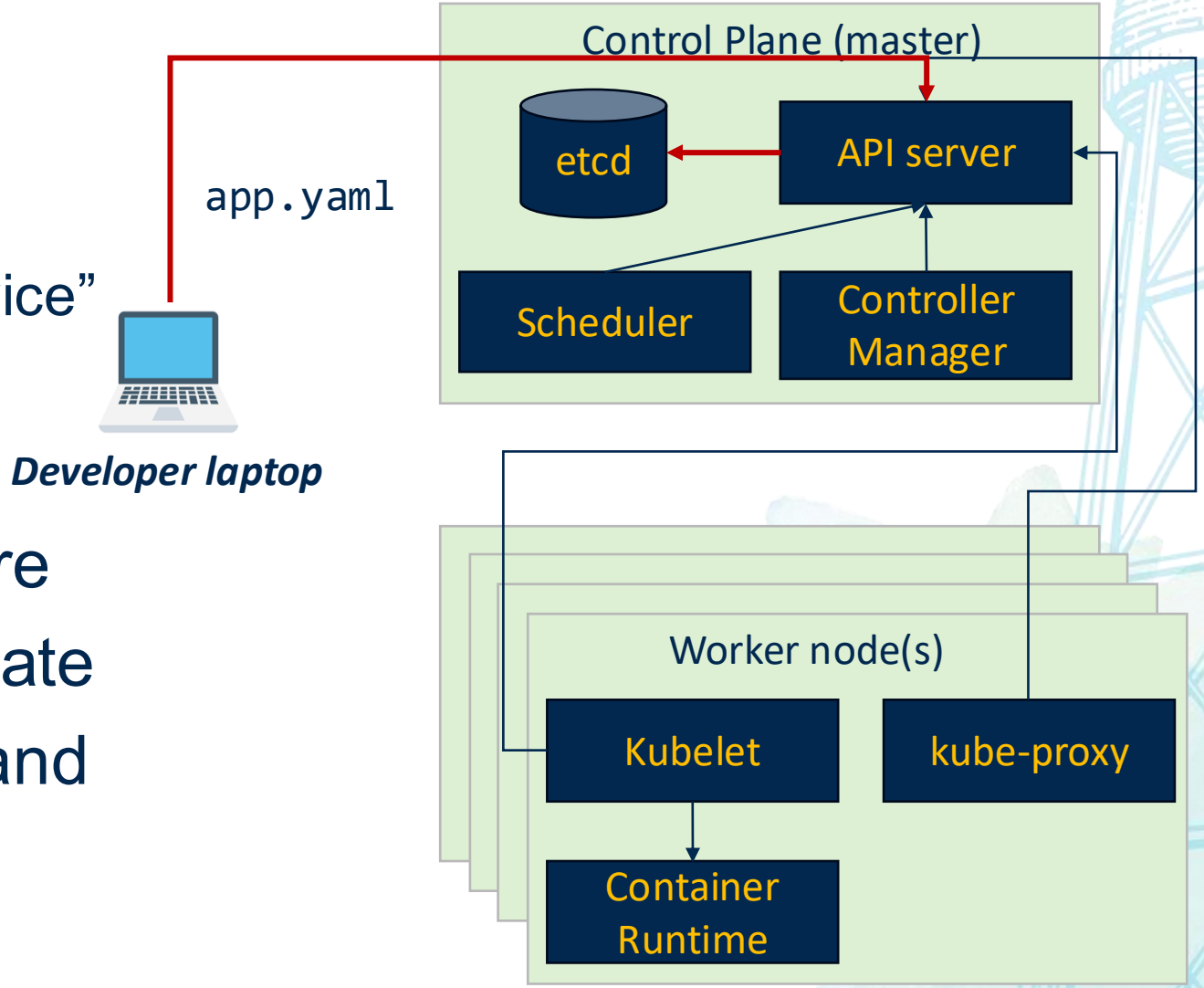
Pod lifecycle

- API server accepts app descriptor YAML files
  - ”Run 3 instances of order-service”
- Stores state in etcd



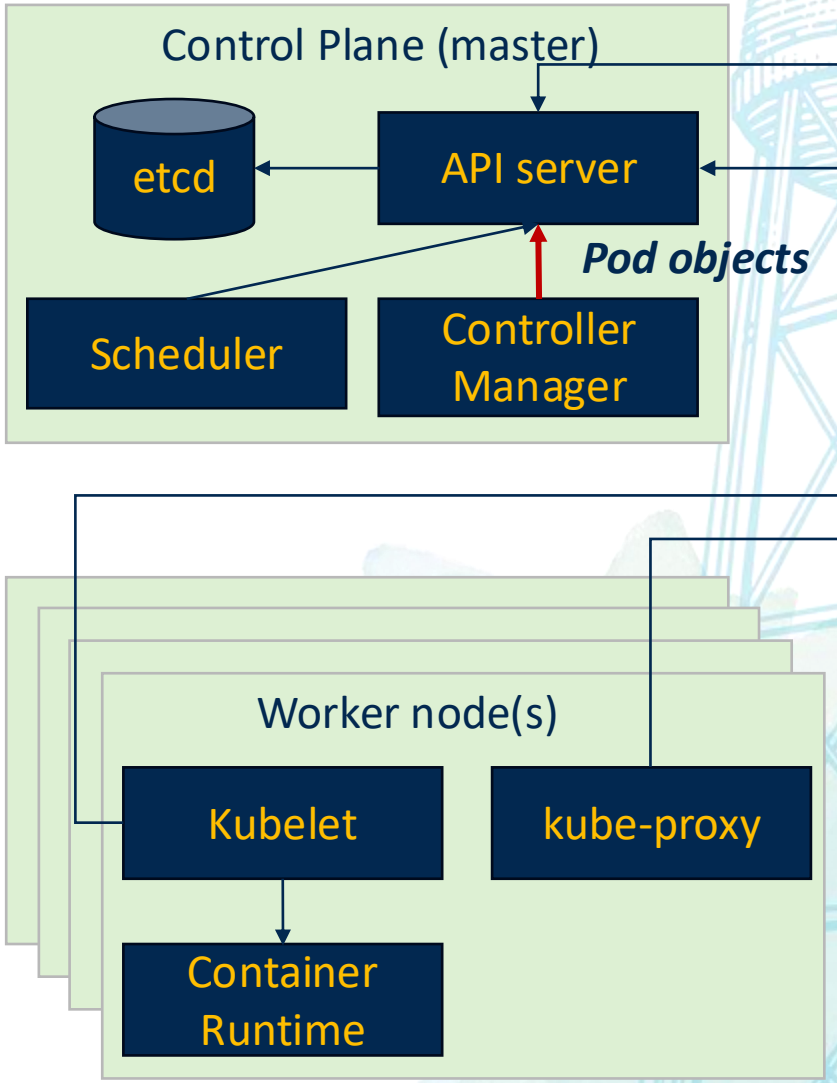
Pod lifecycle

- API server accepts app descriptor YAML files
  - “Run 3 instances of order-service”
- Stores state in etcd
- etcd is a distributed KV-store used to persist and coordinate the cluster’s configuration and state



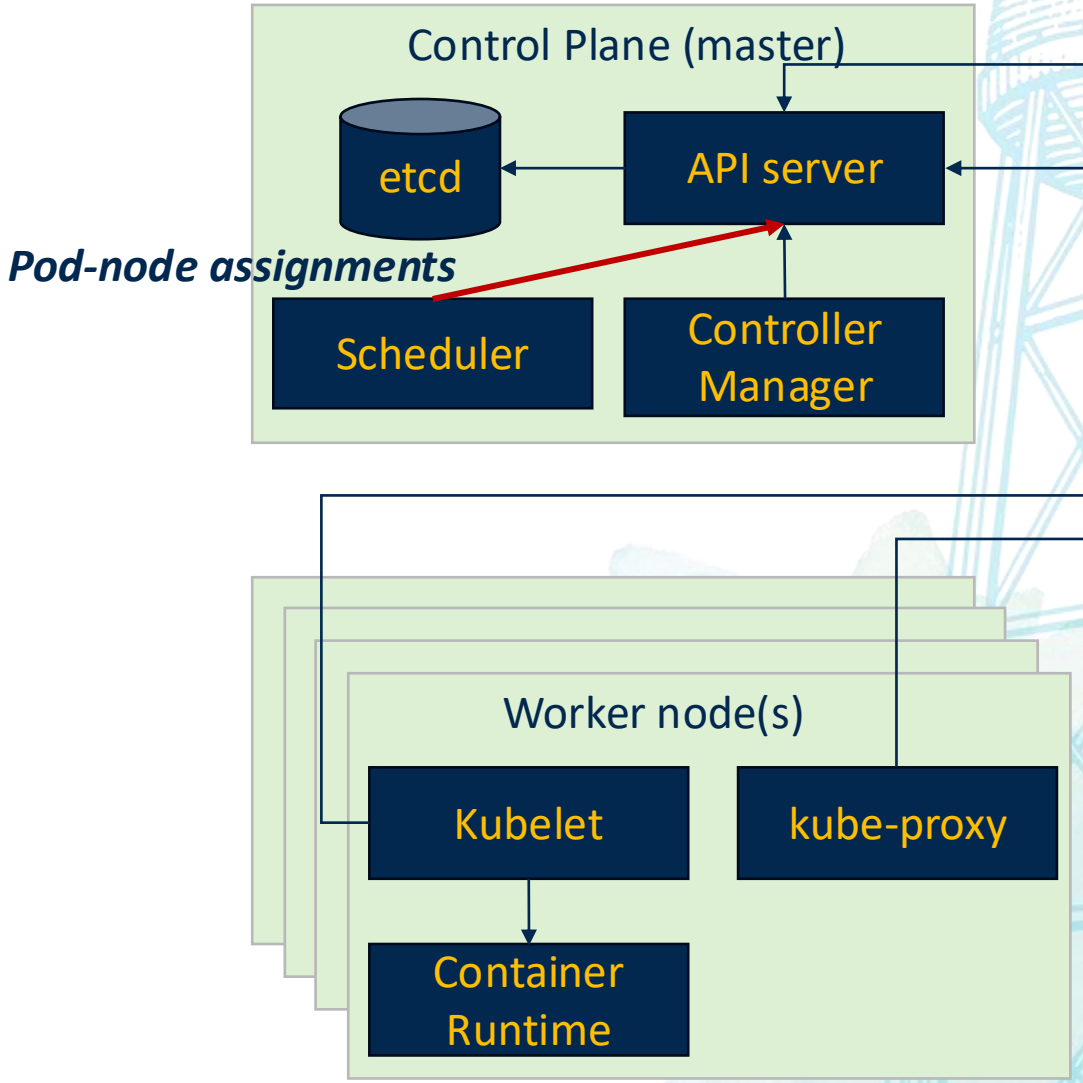
Pod lifecycle

- Controller Manager detects "desired state" != "actual state"
- Creates pod objects in the API server



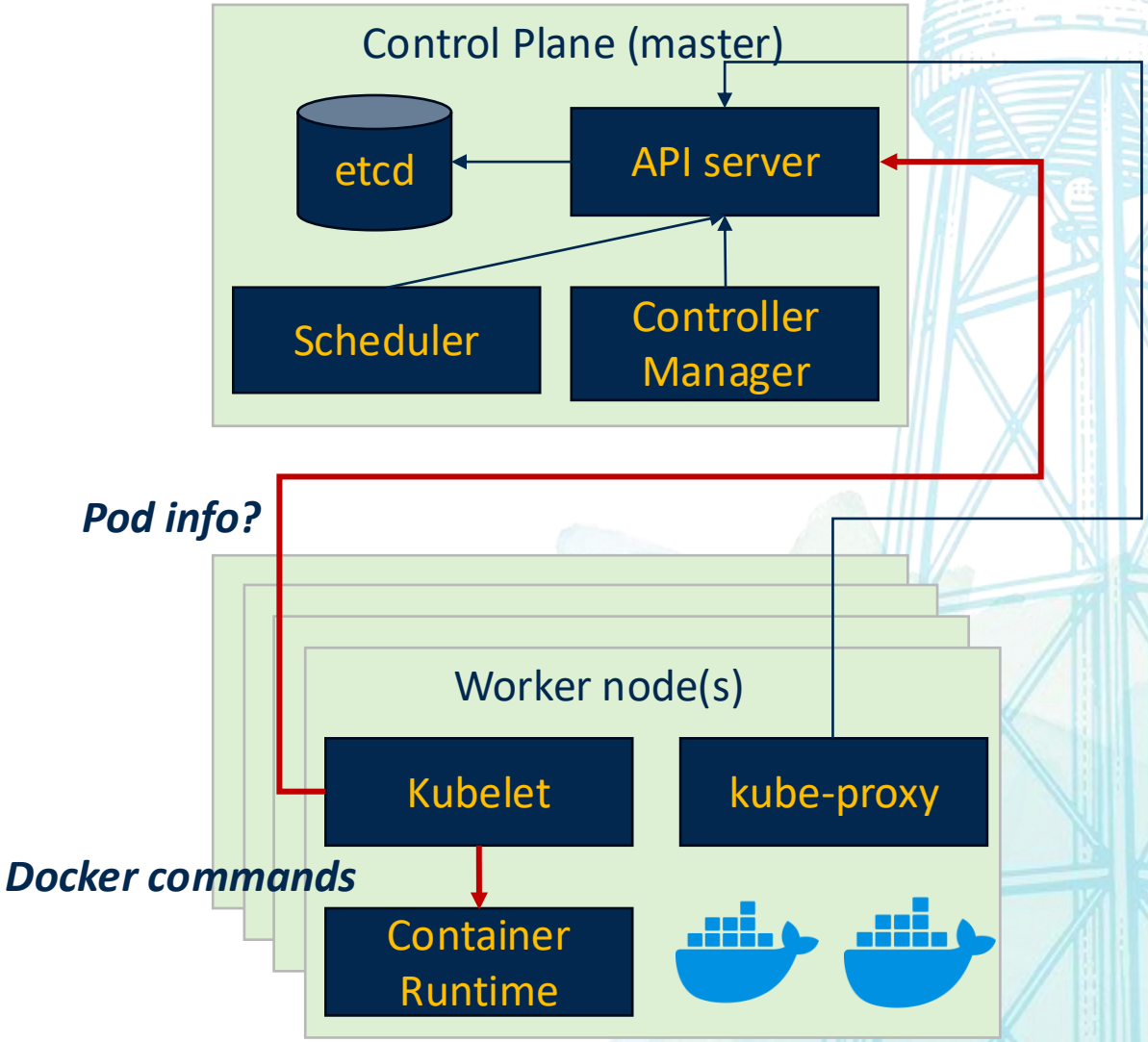
Pod lifecycle

- Scheduler watches for unassigned pods on the API server
- Picks a suitable worker node for each pod
- Pushes the assignment to the API server



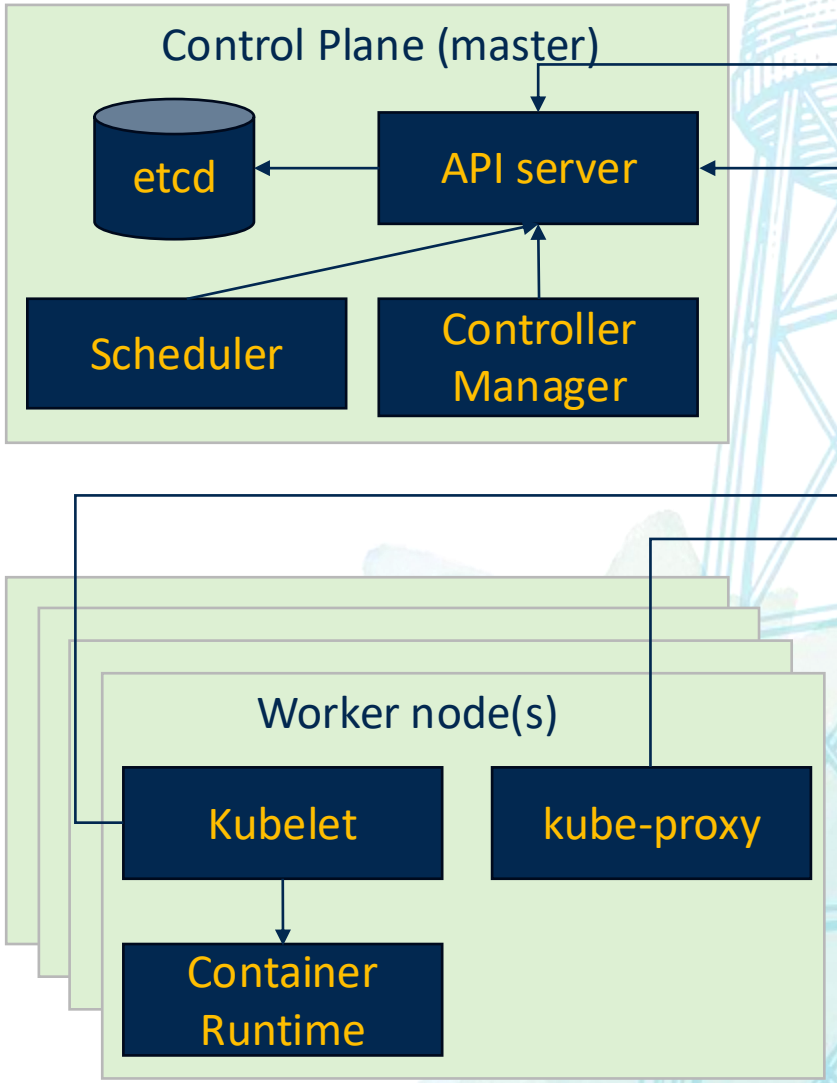
Pod lifecycle

- Kubelets running on worker nodes request the API server for pods that are scheduled on their nodes
- Instruct the container runtime (docker) to download and launch the containers



Pod lifecycle

- Key design points:
  - Kubernetes components communicate through each other via the API server
  - Each component ***pulls*** the updates



Cluster management requirements

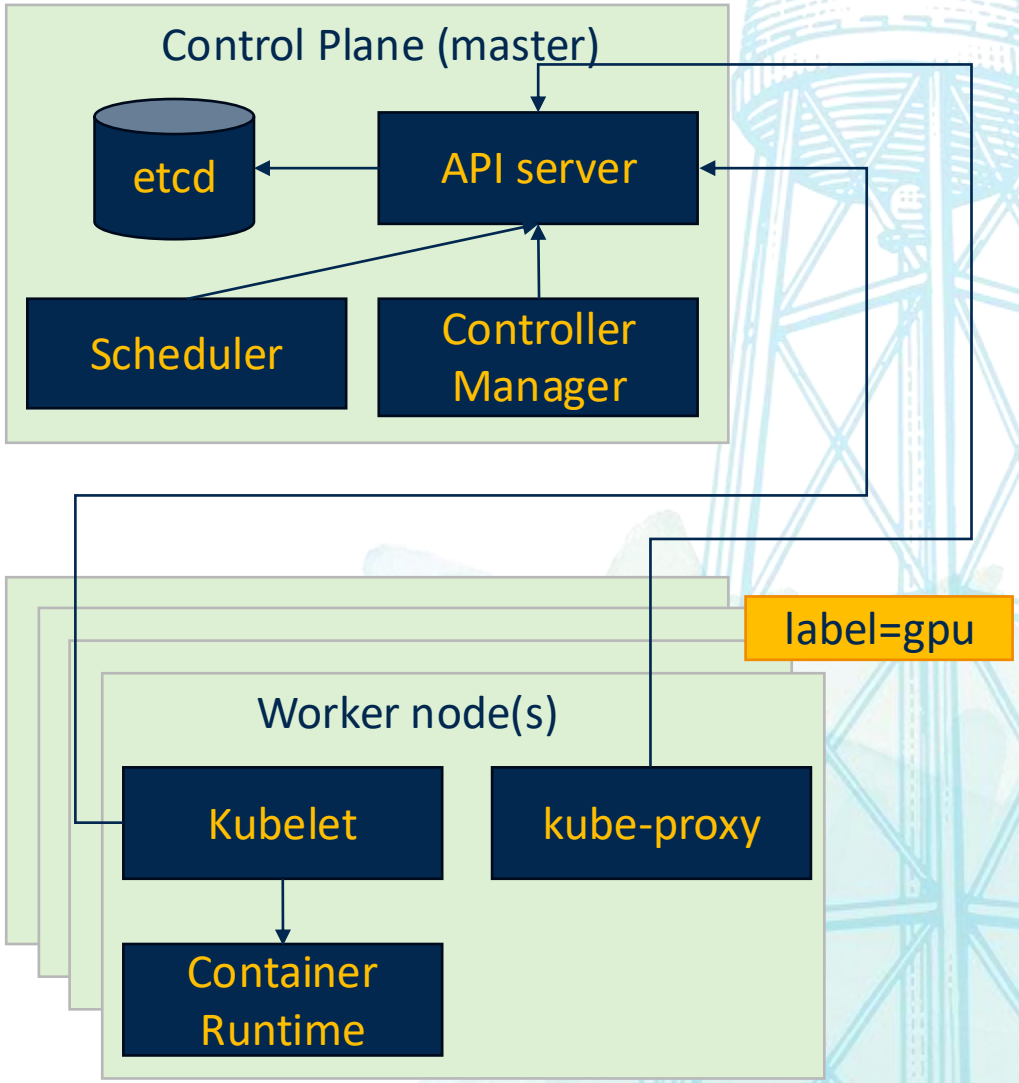
- Automatically schedule services on nodes
  - Each service needs its own environment
- Minimize communication overhead
  - Deploy ”chatty” microservices on the same node
- Ensure the node is well-provisioned to handle the service scheduled on it
  - Heterogenous clusters might have nodes with different CPU/GPU/RAM/disk capabilities
- Detect node failures to stop routing requests to the failed nodes

Label-based scheduling

- Kubernetes allows assigning labels to objects (pods, worker nodes)
- Labels = key-value pairs attached to object (e.g., `app=frontend`, `env=production`)
- Label selectors
  - Match on exact key-value: `env=production`, `app!=frontend`
- Scheduler assigns pod to a node that satisfies the selector constraints
- Example: schedule GPU workloads only on `gpu=true` nodes

Label-based scheduling

```yaml
apiVersion: apps/v1
metadata:
  name: order-service
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: order-service
          image: your-registry/orderservice:latest
      nodeSelector:
        - gpu: "true"
```

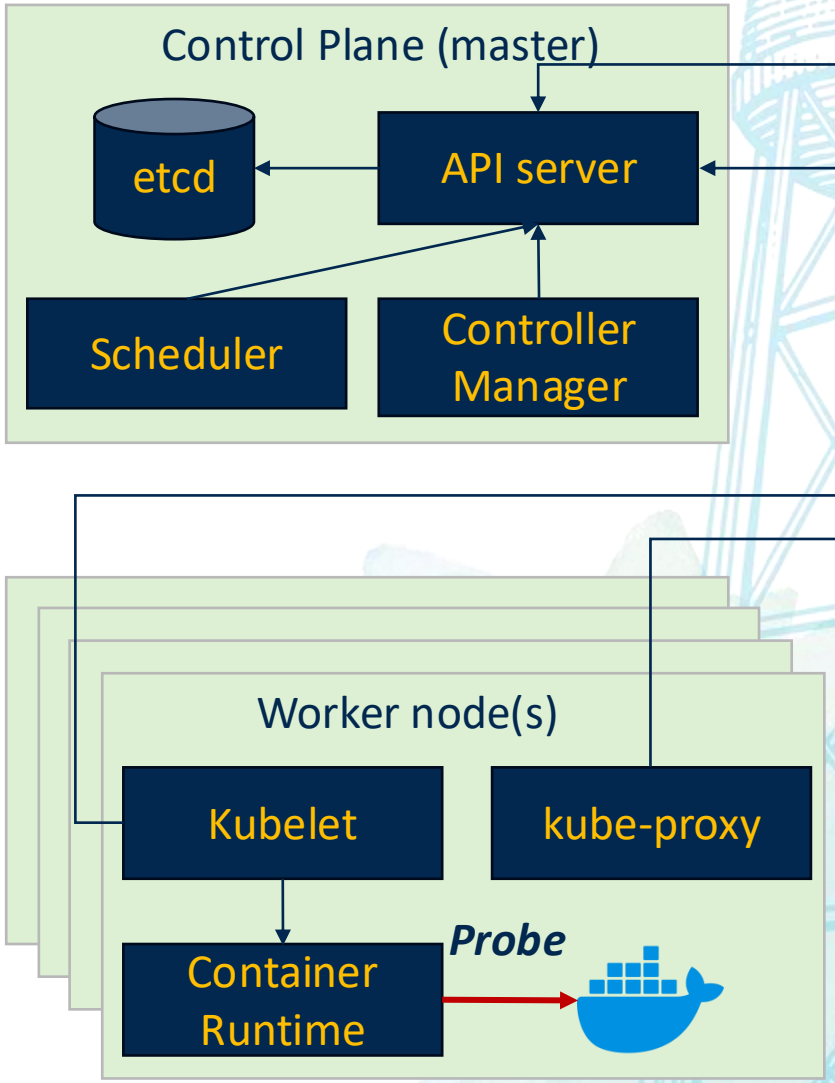


Fault tolerance

- When master node dies and reboots, it restores state from etcd
- When container crashes (e.g., OOM, unhandled exception), Kubelet automatically restarts it
- When a node goes down, Controller Manager creates replacement pods on healthy nodes
- What if container is running, but not responding?
  - Deadlocked, infinite loop, etc..

Liveness probes

- Can configure per-pod liveness probes
- Kubelet will probe the container for liveness signs
- Liveness probes defines health checks
  - HTTP GET — hit an endpoint, expect 200
  - TCP socket — check if a port is open
  - Exec — run a command inside the container, expect exit code 0
- If the probe fails repeatedly, Kubelet kills and restarts the container

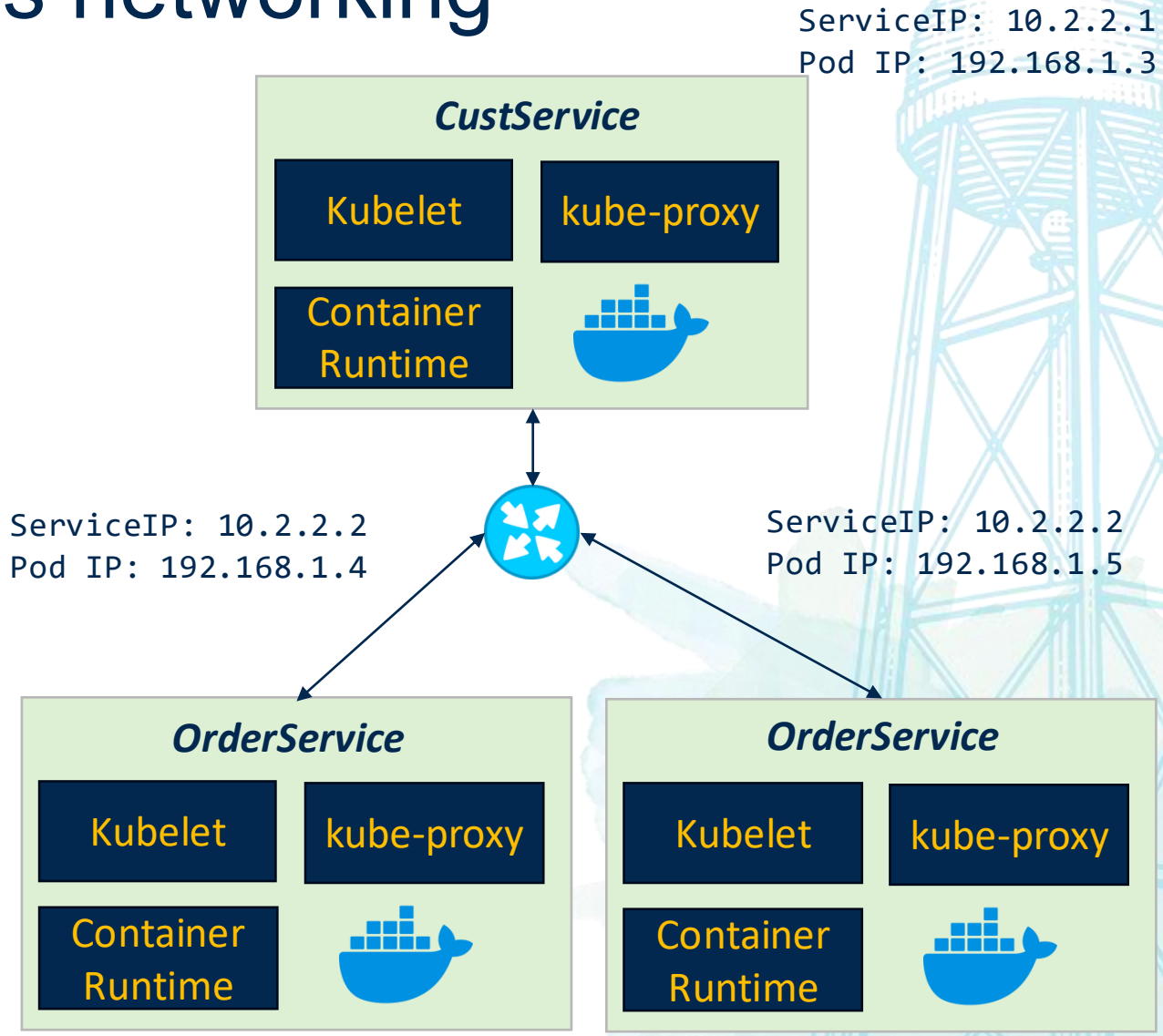


Cluster management requirements

- Automatically schedule services on nodes
  - Each service needs its own environment
- Minimize communication overhead
  - Deploy ”chatty” microservices on the same node
- Ensure the node is well-provisioned to handle the service scheduled on it
  - Heterogenous clusters might have nodes with different CPU/GPU/RAM/disk capabilities
- Detect node failures to stop routing requests to the failed nodes

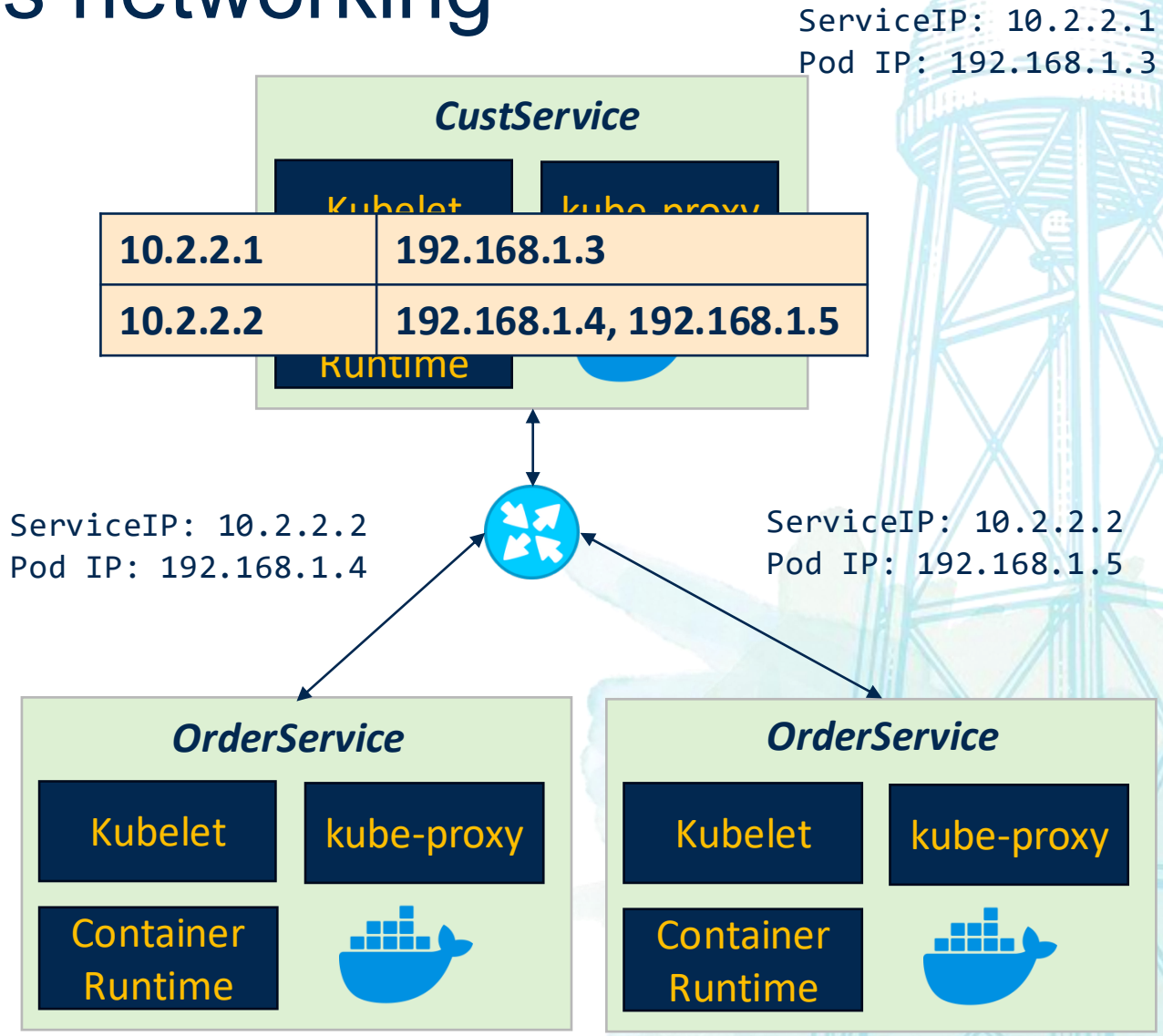
Kubernetes networking

- Services have a logical “Service IP”
- All pod replicas share the same Service IP

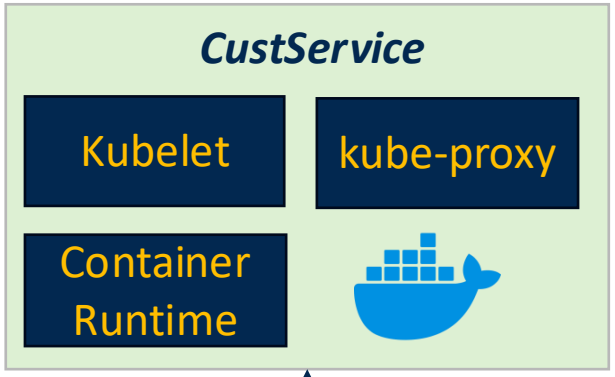


Kubernetes networking

- `kube-proxy` maintains a list of Service IP to pod IP mappings
- Rewrites outgoing Service IPs to the pod IP



Kubernetes networking

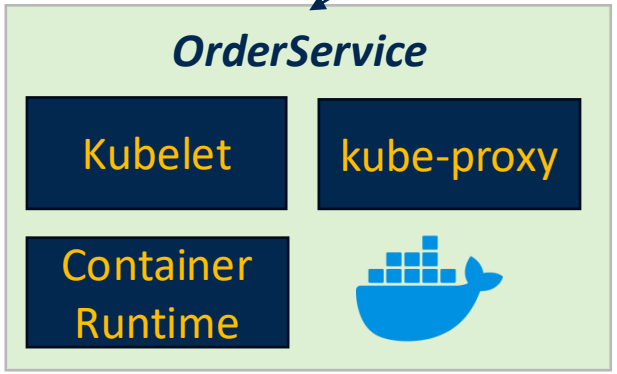


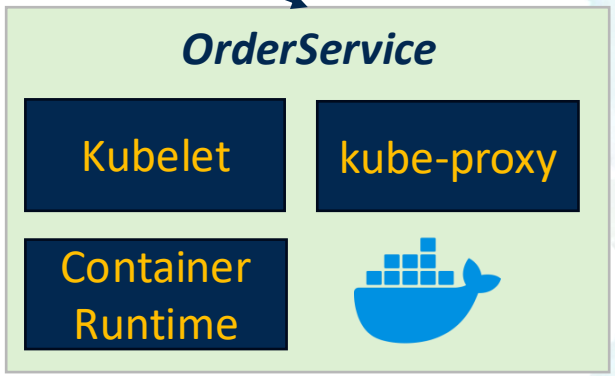
ServiceIP: 10.2.2.1
Pod IP: 192.168.1.3

10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

ServiceIP: 10.2.2.2
Pod IP: 192.168.1.4

ServiceIP: 10.2.2.2
Pod IP: 192.168.1.5

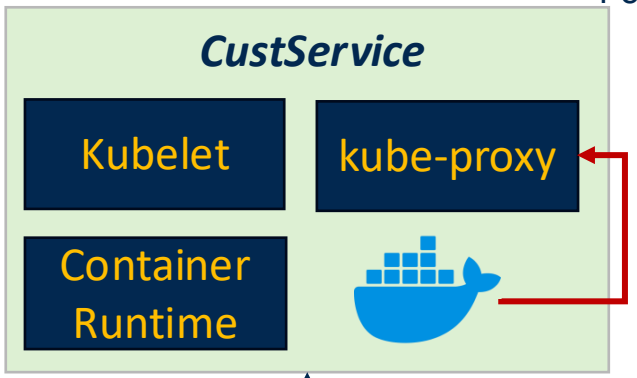




10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

Kubernetes networking



ServiceIP: 10.2.2.1
Pod IP: 192.168.1.3

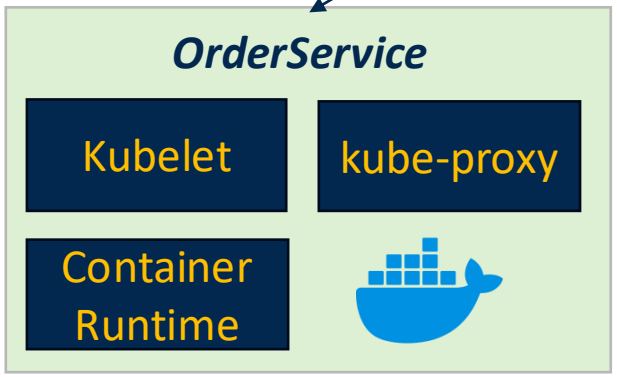
10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

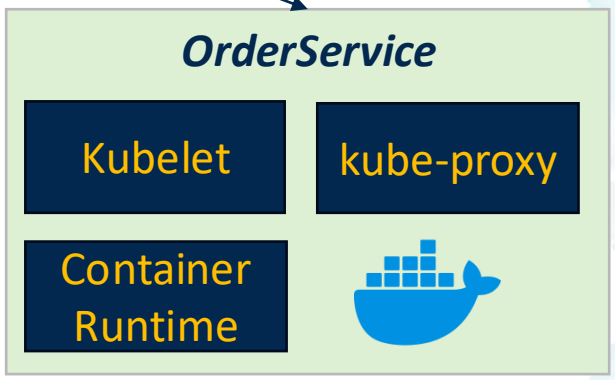
Dest IP addr: 10.22.2.2

ServiceIP: 10.2.2.2
Pod IP: 192.168.1.4

ServiceIP: 10.2.2.2
Pod IP: 192.168.1.5

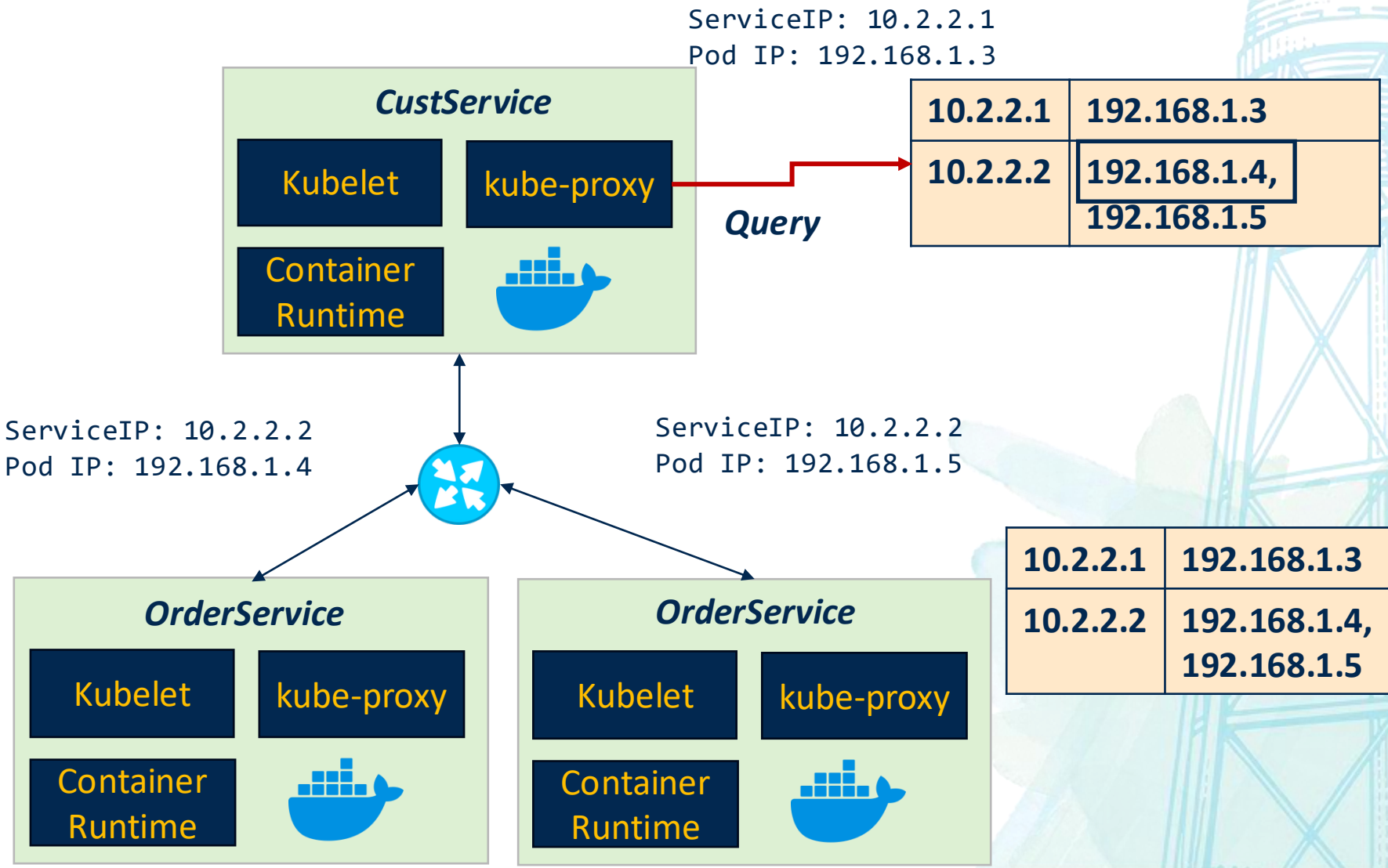
10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5





10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

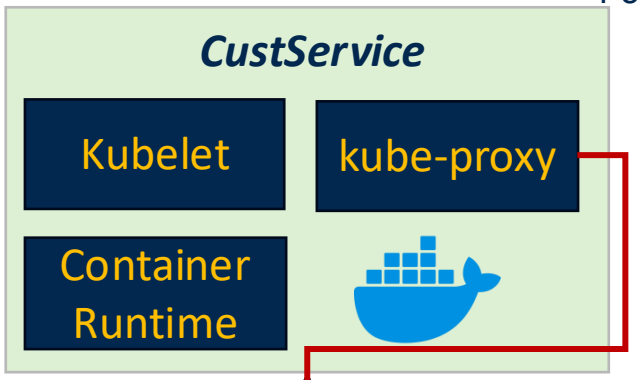
Kubernetes networking



10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

Kubernetes networking

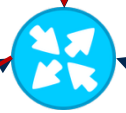


ServiceIP: 10.2.2.1
Pod IP: 192.168.1.3

10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

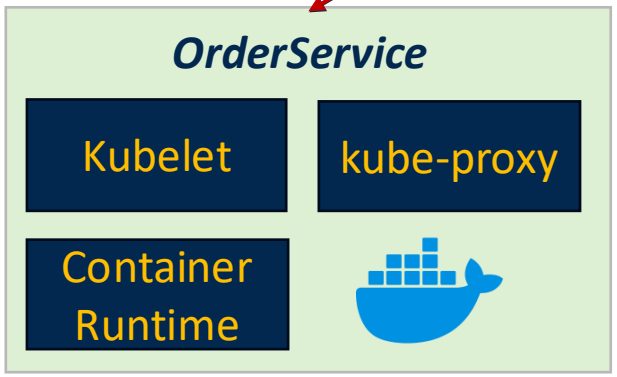
Dest IP addr: 192.168.1.4

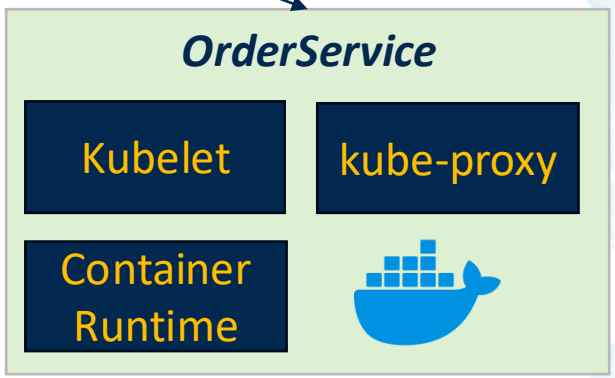
ServiceIP: 10.2.2.2
Pod IP: 192.168.1.4



ServiceIP: 10.2.2.2
Pod IP: 192.168.1.5

10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5





10.2.2.1	192.168.1.3
10.2.2.2	192.168.1.4, 192.168.1.5

Additional features

- Autoscaling: supports HPA (Horizontal Pod Autoscaler)
  - Monitors CPU/memory utilization
  - Increases the replica count when threshold reached
- Custom network fast-paths
  - CNI controls pod networking + routing
  - eBPF-based CNIs can avoid iptables for lower overhead

Kubernetes summary

- Kubernetes automates deployment, scaling, and management of containerized applications across clusters of machines
- Pods are the unit of deployment and scaling
- Control plane manages desired state, and the worker nodes execute them
- Has built-in fault tolerance and resilience